

Recurrent Neural Networks

No _____

Date _____

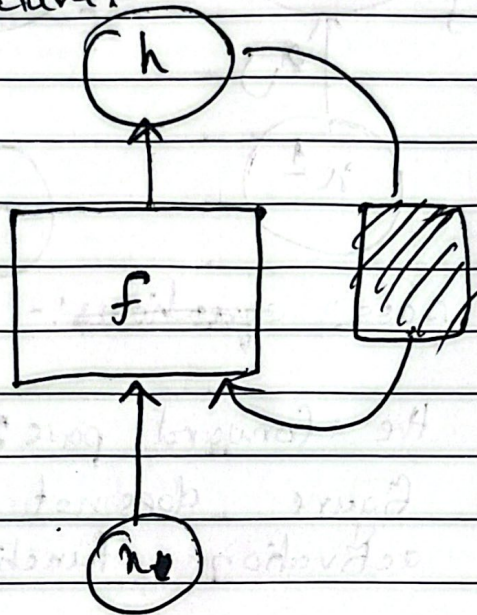
Problem Context:-

$$\text{Input} = (x_1, x_2, x_3)$$

$$\text{Hidden state function, } h_t = f(x_t, h_{t-1})$$

Scalar loss is calculated at the final step as,
$$L = L(h_3)$$

RNN Architecture:-



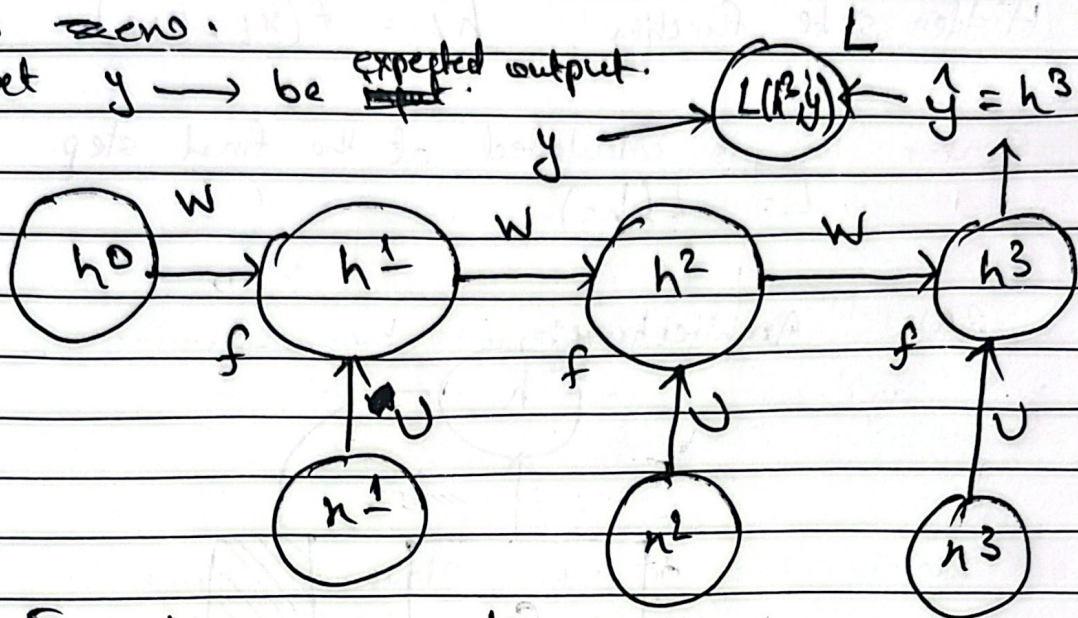
This is a RNN with a hidden-state as output, the shaded box represents a time delay.

This RNN is unfolded, ~~use can~~ and it is difficult for us to visualize how the RNN actually processes the data, we can ~~unfold~~ this RNN as follow,

RNN unfolded for $(n_1, n_2 \& n_3)$ with the last hidden state as the output.

let $h^0 \rightarrow$ be a hidden state initialized to zero.

let $y \rightarrow$ be ~~input~~ expected output.



1. Forward pass: ~~equations~~:-

During the forward pass:

The above figure does not assume any specific choice of activation functions

Let us assume a weight " U " assigned to the connections from input nodes x^t to the hidden nodes h^t , the weight " U " is shared for all the time steps.

Also, let weight " W " be assigned to nodes from previous hidden step to the current hidden step, the weight " W " is also shared,

The sharing of weights " W " & " U " ~~across~~ across time allows the model to generalize for any number of sequence a different parameter ~~would~~ for each time step would mean RNN would not be able to adapt for varying sequence lengths.

Assume, the ~~input $\rightarrow$ hidden~~ activation functions in the hidden state is " σ ", σ is any general activation function but usually hyperbolic tangent, tanh is used.

With this function, we can calculate the ^{linear} activation ~~in~~ in hidden state as,

$$q(t) = b + W h^{(t-1)} + U h^{(t)}$$

the linear activation $a^{(t)}$ is then passed into a activation function σ , to calculate the value for current $h^{(t)}$

$$\text{i.e. } h^{(t)} = \sigma(q^{(t)})$$

$$\text{Or, in more practical settings, } h^{(t)} = \tanh(a^{(t)})$$

~~as $h^{(t)}$ can be recursively expanded to~~ as $h^{(t)}$ is a function of $h^{(t-1)}$, $h^{(t)}$ is a recursive function and when expanded. ~~as~~ upto the initial state, we can see it carries information from the very beginning of the sequence.

2, The hidden state dynamics;

We have the equations,

$$a(t) = b + W h(t-1) + U n(t) \quad - (i)$$

$$h(t) = \sigma(a(t)) \quad - (ii)$$

from this equation we clearly see,

$h(t)$ is a function of $h(t-1)$ & $n(t)$

if we recursively expand $h(t)$, we see

~~$h(t)$~~

that $h(t)$ stores the information from $h^{(t-1)}$ to h^0 ,

The hidden state's contribution is controlled by the shared weight " W ",

During the training process, the weight W is updated to generalize how the previous hidden states interact with the next ones,

In this way, in RNN, hidden states accumulate the informations from previous steps.

3. Back-propagation Through Time:

Standard back propagation used in feed-forward NN or MLPs ~~only~~ is used to propagate the errors backwards in the layer to update the parameters, in those networks gradients are calculated for each ~~training step~~ data in training step. It is possible to perform back propagation as there is a fixed ~~sequence~~ length of data.

RNN have a different story, RNN can process any sequence of data & the parameters that are to be updated are shared for all the steps in the sequence, Further more, the shared parameters need to be updated based on the whole sequence of activations. as the calculations are recursively linked.

To make the calculation of gradients possible, during back-propagation, the RNN is unfolded and each of the activations in previous time steps are stored in caches.

or loss

The error for our particular case is computed at the final hidden state as $L(h^{(t)}) \Rightarrow$ This error is propagated backwards through each time step and a final gradient update for the shared parameters are computed.

As a shared parameter is updated by ~~backpropagating~~ propagating the error through time, this algorithm is called back propagation through time.

4. Chain rule derivation for $\frac{\partial L}{\partial h^{(1)}}$

$$a^{(2)} = b + W h^{(1)} + U n^{(1)}$$

$$h^{(2)} = \sigma(a^{(2)})$$

$$L = L(h^{(3)})$$

$h^{(1)}$ affects L through the chain.

$$h^{(1)} \rightarrow a^{(2)} \rightarrow h^{(2)} \rightarrow a^{(3)} \rightarrow h^{(3)} \rightarrow L$$

Chain-Rule Expansion:-

~~$$\frac{\partial L}{\partial h^{(1)}} = \frac{\partial a^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(2)}}{\partial a^{(2)}} \frac{\partial a^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(3)}}{\partial a^{(3)}} \frac{\partial L}{\partial h^{(3)}}$$~~

$$\frac{\partial L}{\partial h^{(1)}} = \left(\frac{\partial a^{(2)}}{\partial h^{(1)}} \right) \left(\frac{\partial h^{(2)}}{\partial a^{(2)}} \right) \left(\frac{\partial a^{(3)}}{\partial h^{(2)}} \right) \left(\frac{\partial h^{(3)}}{\partial a^{(3)}} \right) \frac{\partial L}{\partial h^{(3)}}$$

Linear activation derivatives.

$$\frac{\partial a^{(2)}}{\partial h^{(1)}} = W$$

$$\frac{\partial a^{(3)}}{\partial h^{(2)}} = U$$

Non-linear activation derivatives:-

$$\frac{\partial h^{(2)}}{\partial a^{(2)}} = \text{diag}(\sigma'(a^{(2)}))$$

$$\frac{\partial h^{(3)}}{\partial a^{(3)}} = \text{diag}(\sigma'(a^{(3)}))$$

$$\frac{\partial L}{\partial h^{(1)}} = W^T \left[\text{diag}(\sigma'(a^{(2)})) \right]^T W^T \left[\text{diag}(\sigma'(a^{(3)})) \right]^T \times \frac{\partial L}{\partial h^{(3)}}$$

$$\therefore \frac{\partial L}{\partial h^{(1)}} = W^T \text{diag}(\sigma'(a^{(2)})) W^T \text{diag}(\sigma'(a^{(3)})) \frac{\partial L}{\partial h^{(3)}}$$

$$= W^T \odot \sigma'(a^{(2)}) W^T \odot \sigma'(a^{(3)}) L'(h^3),$$

Recursive view,

$$\nabla_{h^{(3)}} L = \frac{\partial L}{\partial h^{(3)}}$$

$$\nabla_{h^{(2)}} L = W^T \text{diag}(\sigma'(a^{(3)})) \nabla_{h^{(3)}} L$$

$$\nabla_{h^{(1)}} L = W^T \text{diag}(\sigma'(a^{(2)})) \nabla_{h^{(2)}} L$$

5) Intermediate activations & Memory:

~~For~~ To update the weights & biases, we need to store the value of each activation (h_1, h_2, h_3) in hidden state, to update the weights, these activations are needed.

These activations are stored in an ordered list of ~~active~~ caches,

For a sequence of n inputs, the storage needs scale linearly by $O(n)$.

These are stored because backpropagation can only be performed after each time step in sequence is processed.

6) Parameter Sharing:-

Parameter sharing is done for following reasons:-

a) With multiple parameters for each time step RNN will not be able to work on varying sequence

b) The storage cost will grow if parameters are not shared,

c) RNN need to generalize on both placement of inside a sequence and sequence length,

sharing parameter enables all these.

To update the parameter;

The gradients from each ~~time step~~ ^{time step} the ~~sequence~~ ^{sequence} parameter is used in needs to be summed together.

$$\text{i.e. } \frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L}{\partial a^{(t)}} \frac{\partial a^{(t)}}{\partial W},$$

Vanishing Gradient & Exploding Gradients.

The term,

$$\frac{\partial L}{\partial a^{(t)}} \frac{\partial a^{(t)}}{\partial W} \quad \text{let } t=1, \text{ then,}$$

$$\begin{aligned} \frac{\partial L}{\partial h^{(1)}} \frac{\partial h^{(1)}}{\partial a^{(1)}} &= \frac{\partial L}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial W} \\ &= \left[\frac{\partial L}{\partial h^{(1)}} \frac{\partial h^{(1)}}{\partial a^{(1)}} \right] \frac{\partial a^{(1)}}{\partial W} \end{aligned}$$

This part as we saw before is expanded to upto the last time sequence & there is a term,

calculation, $\prod_{t=2}^T W^t \odot \sigma'(a^{(t)})$ in the

This, term long chain of multiplication when performed for large sequence T , can severely increase or decrease the gradients, causing the weight updates to be very less or very dramatic.